

LogGuard V2

Reporte Técnico de Autómatas Finitos No Deterministas Detección de Amenazas en Logs de PostgreSQL

Este documento presenta la descripción formal del lenguaje de patrones de ataque reconocido por LogGuard V2, incluyendo la definición del sistema de autómatas finitos no deterministas, diagramas de estados, tablas de transiciones, decisiones de diseño y análisis de cadenas de prueba.

Autor: Adolfo Toledo

Versión: 2.1.0

Fecha: Junio 2026

Plataforma: .NET 10.0 / Windows / WPF

Tecnología: Autómata Finito No Determinista (NFA)

Índice general

1. Descripción del Lenguaje	1
1.1. Contexto del Sistema	1
1.2. El Lenguaje de Entrada	1
1.2.1. Capa 1 — Formato de Log de PostgreSQL	1
1.2.2. Capa 2 — Lenguaje de Tokens SQL	2
1.3. Pipeline de Procesamiento	2
1.4. Fases del Tokenizador	3
2. Definición Formal	4
2.1. Autómata Finito No Determinista (NFA)	4
2.2. Perfil de Amenaza	4
2.3. Algoritmo de Simulación	5
2.4. Inventario de Perfiles NFA	5
3. Diagramas de Autómatas	6
3.1. NFA — Inyección SQL (<code>pgsql-sqli-v2</code>)	6
3.2. NFA — Escalada de Privilegios	6
3.3. NFA — Exfiltración de Datos (<code>pgsql-exfil-v1</code>)	7
4. Tablas de Transiciones	8
4.1. SQL Injection — <code>pgsql-sqli-v2</code>	8
4.2. Privilege Escalation — <code>pgsql-privesc-v1</code>	9
4.3. Data Exfiltration — <code>pgsql-exfil-v1</code>	9
5. Decisiones de Diseño Justificadas	11
5.1. Uso de NFA en lugar de expresiones regulares simples	11
5.2. Tokenización en cuatro fases con marcador TAUTOLOGY	11
5.3. Modo de reconocimiento de subcadena (<i>substring match</i>)	12
5.4. Restricción negativa (<code>requireAbsentTokens</code>)	12
5.5. Arquitectura de pipeline asíncrono	12
5.6. Evaluación de perfiles en formato JSON con tipado fuerte	13
6. Cadenas de Prueba	14
6.1. Pruebas sobre SQL Injection (<code>pgsql-sqli-v2</code>)	14
6.1.1. Caso P1 — Tautología directa	14
6.1.2. Caso P2 — UNION Injection	14
6.1.3. Caso P3 — Consulta legítima (no debe activar alarma)	15
6.2. Pruebas sobre Privilege Escalation	15
6.2.1. Caso P4 — Escalada directa	15

6.2.2. Caso P5 — Con cláusula WITH	16
6.3. Pruebas sobre Data Exfiltration	16
6.3.1. Caso P6 — Volcado masivo	16
6.3.2. Caso P7 — SELECT * legítimo con WHERE	16
6.4. Resumen de Cadenas de Prueba	16
7. Conclusiones	17
7.1. Síntesis del sistema	17
7.2. Fortalezas del diseño	17
7.3. Limitaciones y trabajo futuro	18
7.4. Conclusión final	18

Capítulo 1

Descripción del Lenguaje

1.1. Contexto del Sistema

LogGuard V2 es una aplicación de seguridad en tiempo real que monitorea archivos de registro (*logs*) de instancias PostgreSQL. Su objetivo es detectar comportamientos maliciosos en consultas SQL antes de que causen daño. El sistema lee logs en formato estándar de PostgreSQL, extrae las sentencias SQL, las tokeniza y aplica un conjunto de Autómatas Finitos No Deterministas (NFA) para clasificar cada consulta como *benigna* o *amenaza*.

1.2. El Lenguaje de Entrada

El sistema opera sobre dos capas de lenguaje:

1.2.1. Capa 1 — Formato de Log de PostgreSQL

Las líneas de log siguen una de dos plantillas de prefijo configurables en `postgresql.conf` mediante el parámetro `log_line_prefix`:

Estilo	Prefijo	Ejemplo
%m	%m [%p] %q%u@%h%d	2026-06-21 20:10:15.123 UTC [1234] app@10.0.0.1 mydb
%t	%t [%p]: [%l-%s] user=%u,...	2026-06-21 20:10:15 UTC [1234]: [1-1] user=app,db=mydb

Cada línea contiene: *timestamp* (ISO 8601 + zona horaria), PID del proceso, nivel de severidad (LOG, ERROR, FATAL, etc.), y el cuerpo del mensaje. Los tipos de línea reconocidos son:

Statement Consulta SQL ejecutada (`statement: <sql>`).

Duration Tiempo de ejecución en ms (`duration: X.Y ms`).

ConnectionReceived Nueva conexión entrante con host y puerto.

ConnectionAuthenticated Método de autenticación usado.

ConnectionAuthorized Sesión autorizada: usuario y base de datos.

Disconnection Cierre de sesión con resumen de duración.

General Mensaje de log sin campos estructurados adicionales.

Unknown Línea no reconocida por ningún patrón.

1.2.2. Capa 2 — Lenguaje de Tokens SQL

Una vez extraída la sentencia SQL del campo `statement:`, el sistema la transforma mediante un proceso de cuatro fases en una *secuencia de tokens*. Esta secuencia es el **lenguaje de entrada formal** para los autómatas.

El alfabeto Σ del lenguaje está compuesto por más de 70 categorías de tokens, agrupadas en:

Categoría	Tokens representativos
Estructura de consulta	<code>SELECT</code> , <code>FROM</code> , <code>WHERE</code> , <code>JOIN</code> , <code>UNION</code> , <code>UNION_ALL</code> , <code>GROUP</code> , <code>ORDER</code>
Comparación lógica	<code>OR</code> , <code>AND</code> , <code>NOT</code> , <code>IN</code> , <code>LIKE</code> , <code>EQUALS</code> , <code>NEQ</code> , <code>GT</code> , <code>LT</code>
DML	<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>MERGE</code> , <code>VALUES</code> , <code>RETURNING</code>
DDL / Privilegios	<code>CREATE</code> , <code>DROP</code> , <code>ALTER</code> , <code>GRANT</code> , <code>REVOKE</code> , <code>SUPERUSER</code>
Funciones peligrosas	<code>SLEEP</code> , <code>BENCHMARK</code> , <code>WAITFOR_DELAY</code> , <code>XP_CMDSHELL</code> , <code>LOAD_FILE</code>
Tablas del sistema	<code>INFORMATION_SCHEMA</code> , <code>PG_SHADOW</code> , <code>PG_USER</code> , <code>PG_ROLES</code>
Especiales	<code>TAUTOLOGY</code> , <code>COMMENT</code> , <code>STAR</code>
Primitivos	<code>STRING</code> , <code>NUMBER</code> , <code>IDENT</code> , <code>SEMICOLON</code> , <code>LPAREN</code> , <code>RPAREN</code>

El token `TAUTOLOGY` no proviene del léxico SQL estándar sino que es generado en la *Fase 2* del tokenizador cuando se detecta una expresión lógicamente siempre verdadera (e.g., `1=1`, `'x'='x'`, `a=a`).

1.3. Pipeline de Procesamiento

El flujo completo desde el archivo de log hasta la alerta es:

1. **Lectura reactiva:** `FileWatcherLive` lee el archivo con sondeo cada 500 ms, detecta rotación por cambio de *inode* y maneja líneas de continuación multi-línea.
2. **Parseo de log:** `PostgreSqlLogParser` aplica 7 expresiones regulares precompiladas para clasificar cada línea y extraer metadatos estructurados.
3. **Tokenización SQL:** `SqlTokenizer` ejecuta 4 fases secuenciales.
4. **Evaluación NFA:** `NfaEngine` ejecuta todos los perfiles NFA habilitados sobre la secuencia de tokens.
5. **Alerta:** Si algún NFA acepta, se genera una entrada en la bitácora de amenazas con nivel de severidad y tipo de ataque.

1.4. Fases del Tokenizador

Fase 1 — Normalización Se eliminan comentarios SQL (`-`, `/* */`), se decodifican literales hexadecimales (`0x41` $\rightarrow$ `A`) y se unifican espacios blancos.

Fase 2 — Detección de tautologías Se aplican 5 expresiones regulares acotadas para identificar expresiones siempre verdaderas y sustituirlas por el marcador `TAUTOLOGY`.

Fase 3 — Escaneo léxico Se recorre el texto normalizado carácter a carácter y se emiten tokens usando un diccionario de 160 entradas y expresiones regulares para literales.

Fase 4 — Fusión Pares de tokens adyacentes se combinan donde corresponde (e.g., `UNION` + `ALL` $\rightarrow$ `UNION_ALL`).

Capítulo 2

Definición Formal

2.1. Autómata Finito No Determinista (NFA)

Definición 2.1 (Autómata Finito No Determinista). Un NFA se define como una 5-tupla

$$\mathcal{M} = (Q, \Sigma, \delta, q_0, F)$$

donde:

- Q es el conjunto finito de estados,
- Σ es el alfabeto finito de símbolos de entrada (tokens SQL),
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ es la función de transición no determinista,
- $q_0 \in Q$ es el estado inicial,
- $F \subseteq Q$ es el conjunto de estados de aceptación.

Definición 2.2 (Lenguaje aceptado). El lenguaje aceptado por $\mathcal{M}$ es

$$L(\mathcal{M}) = \{w \in \Sigma^* \mid \hat{\delta}(\{q_0\}, w) \cap F \neq \emptyset\}$$

donde $\hat{\delta}$ es la extensión de δ a cadenas.

Definición 2.3 (Subcadena aceptada — modo substring). LogGuard opera en modo de reconocimiento de *subcadena*: el estado inicial q_0 se reinyecta al conjunto activo en cada paso, de modo que el autómata detecta el patrón en cualquier posición dentro de la secuencia de tokens:

$$\text{match}(w) = \exists 0 \leq i \leq j \leq |w| : \hat{\delta}(\{q_0\}, w[i..j]) \cap F \neq \emptyset$$

2.2. Perfil de Amenaza

Cada perfil NFA en LogGuard amplía la definición estándar con dos elementos:

Definición 2.4 (Perfil NFA con restricción negativa). Un perfil NFA extendido es una 7-tupla

$$\mathcal{P} = (Q, \Sigma, \delta, q_0, F, T, \text{absent})$$

donde:

- T es el tipo de amenaza (SQLI, EXFIL, PRIVESC, BRUTEFORCE, DISCOVERY),
- $absent \subseteq \Sigma$ es el conjunto de tokens cuya *presencia* en w invalida la aceptación.

La aceptación final se define:

$$\text{accept}(w) = [\hat{\delta}(\{q_0\}, w) \cap F \neq \emptyset] \wedge \forall a \in \text{absent} : a \notin w$$

2.3. Algoritmo de Simulación

El algoritmo implementado en `NfaEngine.Run` es una simulación de subconjuntos incremental de complejidad $O(|w| \cdot |Q|)$:

```

1 HashSet<string> active = new() { startState };
2
3 foreach (string token in tokens)
4 {
5     var next = new HashSet<string> { startState }; // modo substring
6
7     foreach (string state in active)
8         if (_delta.TryGetValue((state, token), out var targets))
9             next.UnionWith(targets);
10
11     active = next;
12
13     if (active.Overlaps(_acceptStates))
14         if (!tokens.Intersect(profile.RequireAbsentTokens).Any())
15             return true;
16 }
17 return false;

```

Listing 2.1: Núcleo del motor NFA (NfaEngine.cs)

2.4. Inventario de Perfiles NFA

LogGuard incluye 6 perfiles NFA cargados desde archivos JSON en tiempo de ejecución mediante `NfaLoader`:

Perfil	ID	Estados	Transiciones	Severidad
SQL Injection	pgsql-sqli-v2	10	29	High
Data Exfiltration	pgsql-exfil-v1	5	6	High
Privilege Escal.	pgsql-privesc-v1	6	7	Critical
Discovery/Enum.	pgsql-discovery-v2	4	5	Medium
Brute Force	pgsql-bruteforce-v1	8	8	Medium
Time-Based SQLi	pgsql-time-sqli-v2	5	6	High

Capítulo 3

Diagramas de Autómatas

Esta sección presenta los diagramas de transición de estados para los tres perfiles más relevantes: SQL Injection, Data Exfiltration y Privilege Escalation.

3.1. NFA — Inyección SQL (pgsql-sqli-v2)

El autómata de inyección SQL reconoce cinco patrones de ataque: tautología directa, bypass OR, UNION injection, funciones de tiempo (SLEEP/BENCHMARK) y enumeración de esquemas (INFORMATION_SCHEMA).

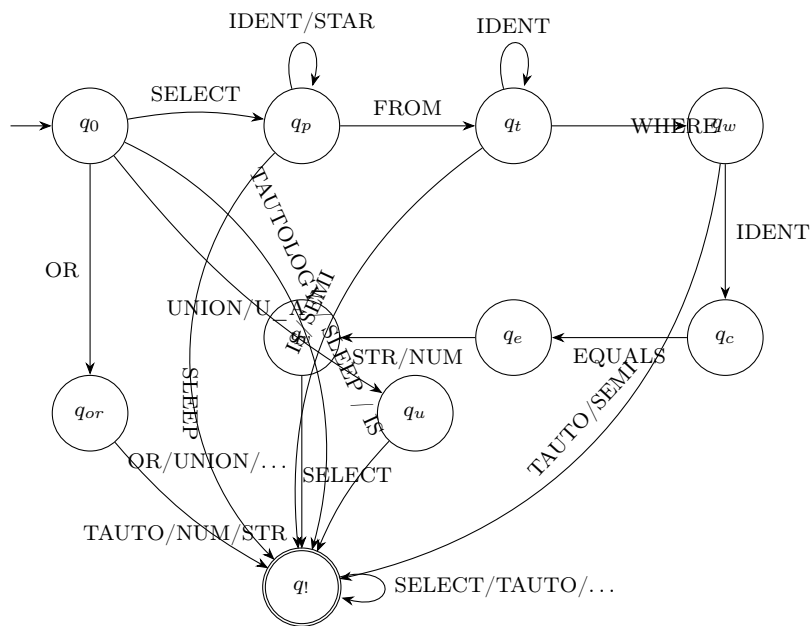


Figura 3.1: NFA para detección de SQL Injection (pgsql-sqli-v2). q_0 =inicial; $q!$ =aceptación; abreviaturas: IS=INFORMATION_SCHEMA, U_A=UNION_ALL, TAUTO=TAUTOLOGY, SEMI=SEMICOLON.

3.2. NFA — Escalada de Privilegios

Perfil: postgresql-privesc-v1.

Este autómata reconoce la sentencia `ALTER USER/ROLE <nombre> [WITH] SUPERUSER`, usada para conceder acceso de superusuario en PostgreSQL. Tiene 6 estados y 7 transiciones, lo que lo hace altamente preciso con mínimos falsos positivos.

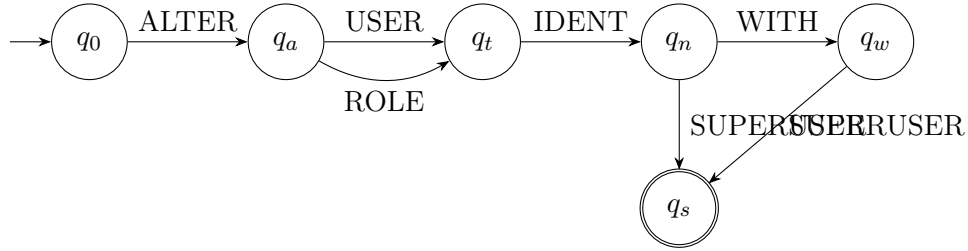


Figura 3.2: NFA para detección de Privilege Escalation (pgsql-privesc-v1). q_0 =inicial; q_s =aceptación (Critical).

3.3. NFA — Exfiltración de Datos (pgsql-exfil-v1)

Detecta volcados masivos: `SELECT */cols FROM tabla` sin cláusula `WHERE` ni `LIMIT`. La restricción negativa (*absent* = {`WHERE`, `LIMIT`}) es el elemento clave.

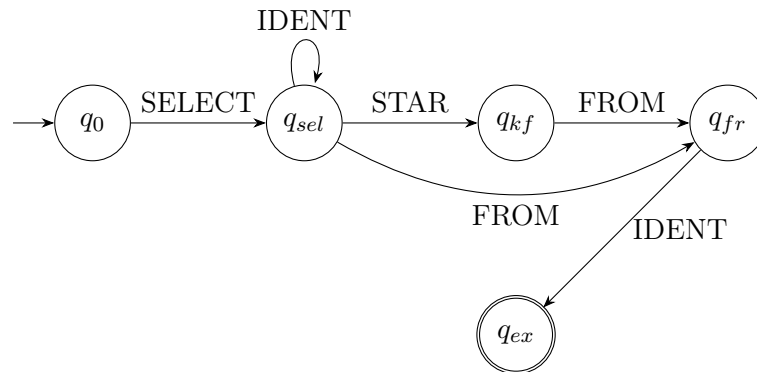


Figura 3.3: NFA para detección de Exfiltración (pgsql-exfil-v1). Aceptación requiere adicionalmente ausencia de `WHERE` y `LIMIT`.

Capítulo 4

Tablas de Transiciones

4.1. SQL Injection — pgsql-sqli-v2

La siguiente tabla presenta la función de transición δ completa del autómata de SQL Injection. Los estados se nombran abreviados para facilitar la lectura.

Cuadro 4.1: Tabla de transiciones — SQL Injection (pgsql-sqli-v2)

Estado origen	Símbolo	Estado destino
q_0	SELECT	q_{prefix}
q_0	TAUTOLOGY	q_{sqli}
q_0	SLEEP	q_{sqli}
q_0	INFORMATION_SCHEMA	q_{sqli}
q_0	OR	q_{or}
q_0	UNION	q_{union}
q_0	UNION_ALL	q_{union}
q_{prefix}	IDENT	q_{prefix}
q_{prefix}	STAR	q_{prefix}
q_{prefix}	FROM	q_{tbl}
q_{prefix}	SLEEP	q_{sqli}
q_{tbl}	IDENT	q_{tbl}
q_{tbl}	INFORMATION_SCHEMA	q_{sqli}
q_{tbl}	WHERE	q_{where}
q_{tbl}	SEMICOLON	q_{sqli}
q_{where}	IDENT	q_{col}
q_{where}	TAUTOLOGY	q_{sqli}
q_{where}	SEMICOLON	q_{sqli}
q_{col}	EQUALS	q_{eq}
q_{eq}	STRING	q_{val}
q_{eq}	NUMBER	q_{val}
q_{val}	OR	q_{sqli}
q_{val}	UNION	q_{sqli}
q_{val}	UNION_ALL	q_{sqli}
q_{val}	TAUTOLOGY	q_{sqli}

Cuadro 4.1: Tabla de transiciones — SQL Injection (continuación)

Estado origen	Símbolo	Estado destino
q_{val}	SLEEP	q_{sqli}
q_{val}	INFORMATION_SCHEMA	q_{sqli}
q_{val}	SEMICOLON	q_{sqli}
q_{or}	TAUTOLOGY	q_{sqli}
q_{or}	NUMBER	q_{sqli}
q_{or}	STRING	q_{sqli}
q_{union}	SELECT	q_{sqli}
q_{union}	ALL	q_{sqli}
q_{sqli}	SELECT	q_{sqli}
q_{sqli}	TAUTOLOGY	q_{sqli}
q_{sqli}	UNION	q_{sqli}
q_{sqli}	UNION_ALL	q_{sqli}
q_{sqli}	SLEEP	q_{sqli}

4.2. Privilege Escalation — pgsql-privesc-v1

Cuadro 4.2: Tabla de transiciones — Privilege Escalation (pgsql-privesc-v1)

Estado origen	Símbolo	Estado destino
q_0	ALTER	q_{alter}
q_{alter}	USER	q_{target}
q_{alter}	ROLE	q_{target}
q_{target}	IDENT	q_{name}
q_{name}	WITH	q_{with}
q_{name}	SUPERUSER	q_{super}
q_{with}	SUPERUSER	q_{super}

4.3. Data Exfiltration — pgsql-exfil-v1

Nota 4.1. Para el perfil de exfiltración, la tabla de transiciones define únicamente las transiciones positivas. La restricción negativa *absent* = {WHERE, LIMIT} se evalúa *post-facto*: si la cadena de tokens contiene alguno de estos dos tokens, la aceptación se cancela aunque el estado q_{ex} sea alcanzado.

Cuadro 4.3: Tabla de transiciones — Data Exfiltration (pgsql-exfil-v1)

Estado origen	Símbolo	Estado destino
q_0	SELECT	q_{sel}
q_{sel}	IDENT	q_{sel}
q_{sel}	STAR	q_{kw_from}
q_{sel}	FROM	q_{from}
q_{kw_from}	FROM	q_{from}
q_{from}	IDENT	q_{ex}

Capítulo 5

Decisiones de Diseño Justificadas

5.1. Uso de NFA en lugar de expresiones regulares simples

Decisión: modelar cada patrón de ataque como un NFA serializado en JSON en lugar de codificar expresiones regulares directamente en el código.

Justificación:

- *Extensibilidad sin recompilación:* agregar un nuevo perfil de amenaza requiere únicamente crear un archivo JSON y copiarlo al directorio `NFA/`. No se modifica ni compila el código C#.
- *Separación de responsabilidades:* el motor (`NfaEngine`) es agnóstico al tipo de amenaza; los analistas de seguridad pueden definir o ajustar perfiles sin conocer el código fuente.
- *Auditabilidad:* el estado del sistema de detección queda explícitamente documentado en archivos de texto (JSON) que pueden versionarse, revisarse y auditarse.
- *Composabilidad:* múltiples perfiles se ejecutan en paralelo sobre la misma secuencia de tokens en $O(|w| \cdot |Q_{\text{total}}|)$.

5.2. Tokenización en cuatro fases con marcador TAUTOLOGY

Decisión: incorporar una fase de pre-análisis semántico (Fase 2) que reemplaza expresiones tautológicas por el token especial **TAUTOLOGY** antes del escaneo léxico.

Justificación:

- Un NFA que intentara reconocer directamente `1=1` o `'x'='x'` necesitaría transiciones para todos los posibles valores numéricos y cadenas, lo que resultaría en un autómata de tamaño exponencial.
- Al abstraer la tautología a un único token, el NFA permanece pequeño y lineal, y el reconocimiento es $O(|w|)$ en lugar de $O(|w| \cdot |\Sigma_{\text{val}}|)$.

- Las 5 expresiones regulares usadas para detectar tautologías están acotadas (longitudes máximas en cuantificadores) para prevenir ataques ReDoS (*Regular expression Denial of Service*).

5.3. Modo de reconocimiento de subcadena (*substring match*)

Decisión: reinyectar q_0 en el conjunto activo en cada paso de simulación, convirtiendo el NFA en un detector de subcadenas.

Justificación:

- Las consultas SQL legítimas rara vez son secuencias puras del patrón de ataque; el payload malicioso suele estar embebido en una consulta más amplia (e.g., `SELECT id FROM t WHERE name='x' OR 1=1`).
- Exigir que el NFA reconozca la cadena *completa* desde q_0 generaría falsos negativos masivos.
- La reinyección de q_0 mantiene la complejidad lineal sin requerir la construcción explícita de un DFA o el uso de búsqueda por sufijos.

5.4. Restricción negativa (`requireAbsentTokens`)

Decisión: añadir un mecanismo de post-condición negativa a los perfiles NFA para invalidar la aceptación cuando ciertos tokens están presentes.

Justificación:

- El perfil de exfiltración (`pgsql-exfil-v1`) detecta `SELECT * FROM tabla`. Sin la restricción negativa, cualquier consulta legítima como `SELECT * FROM config WHERE id=1` sería marcada como amenaza (falso positivo).
- Codificar la ausencia de `WHERE` directamente en el NFA requeriría estados adicionales para cada token que *no* sea `WHERE`, haciendo el autómata exponencialmente complejo.
- La verificación post-facto es $O(|absent|)$ y se realiza una sola vez por evaluación exitosa, sin impacto práctico en el rendimiento.

5.5. Arquitectura de pipeline asíncrono

Componente: `Channel<T>`.

Decisión: desacoplar la lectura de archivos, el parseo y la evaluación NFA mediante un canal productor-consumidor (`System.Threading.Channels`).

Justificación:

- PostgreSQL puede emitir rafagas de cientos de líneas por segundo bajo carga. Un modelo síncrono bloquearía la UI de WPF (que corre en el hilo principal) durante el procesamiento.

- `Channel<T>` proporciona backpressure natural: si el consumidor (NFA engine) va lento, el canal acumula sin perder entradas.
- El modelo *async/await* de .NET 10 permite que la UI siga respondiendo mientras el motor procesa logs en segundo plano.

5.6. Evaluación de perfiles en formato JSON con tipado fuerte

Decisión: serializar los perfiles NFA en JSON con esquema fijo. El tipo `AutomatonProfile` se deserializa con `System.Text.Json`.

Justificación:

- JSON es un formato ubicuo, legible por humanos y soportado nativamente en .NET sin dependencias externas.
- El tipado fuerte (clases `C#` con anotaciones `JsonPropertyName`) detecta errores de configuración en tiempo de carga, no en tiempo de ejecución durante una detección.
- Los archivos JSON se copian al directorio de salida con la política `PreserveNewest`, garantizando que la versión del perfil coincida con la versión del binario.

Capítulo 6

Cadenas de Prueba

En esta sección se presentan cadenas de tokens de prueba con su trayectoria de estados en el NFA correspondiente y el resultado esperado.

6.1. Pruebas sobre SQL Injection (pgsql-sqli-v2)

6.1.1. Caso P1 — Tautología directa

Consulta SQL:

```
SELECT * FROM users WHERE id=1 OR 1=1
```

Secuencia de tokens tras tokenización:

SELECT, STAR, FROM, IDENT, WHERE, IDENT, EQUALS, NUMBER, OR, TAUTOLOGY

Traza de estados:

Paso	Token	Estados activos	Acción
0	—	$\{q_0\}$	inicio
1	SELECT	$\{q_0, q_{\text{prefix}}\}$	$q_0 \xrightarrow{\text{SELECT}} q_{\text{prefix}}$
2	STAR	$\{q_0, q_{\text{prefix}}\}$	loop
3	FROM	$\{q_0, q_{\text{tbl}}\}$	$q_{\text{prefix}} \xrightarrow{\text{FROM}} q_{\text{tbl}}$
4	IDENT	$\{q_0, q_{\text{tbl}}\}$	loop
5	WHERE	$\{q_0, q_{\text{where}}\}$	$q_{\text{tbl}} \xrightarrow{\text{WHERE}} q_{\text{where}}$
6	IDENT	$\{q_0, q_{\text{col}}\}$	$q_{\text{where}} \xrightarrow{\text{IDENT}} q_{\text{col}}$
7	EQUALS	$\{q_0, q_{\text{eq}}\}$	$q_{\text{col}} \xrightarrow{\text{EQUALS}} q_{\text{eq}}$
8	NUMBER	$\{q_0, q_{\text{val}}\}$	$q_{\text{eq}} \xrightarrow{\text{NUMBER}} q_{\text{val}}$
9	OR	$\{q_0, q_{\text{sqli}}, q_{\text{or}}\}$	$q_{\text{val}} \xrightarrow{\text{OR}} q_{\text{sqli}}$
10	TAUTOLOGY	$\{q_0, q_{\text{sqli}}\}$	ACEPTA

Resultado: **AMENAZA** detectada. Tipo: **SQLI**, severidad: High.

6.1.2. Caso P2 — UNION Injection

Consulta SQL:

```
SELECT id FROM t UNION ALL SELECT password FROM pg_shadow
```

Secuencia de tokens:

SELECT, IDENT, FROM, IDENT, UNION_ALL, SELECT, IDENT, FROM, IDENT

Traza abreviada:

Token	Estados activos	Nota
SELECT	$\{q_0, q_{\text{prefix}}\}$	
IDENT	$\{q_0, q_{\text{prefix}}\}$	loop
FROM	$\{q_0, q_{\text{tbl}}\}$	
IDENT	$\{q_0, q_{\text{tbl}}\}$	loop
UNION_ALL	$\{q_0, q_{\text{union}}\}$	$q_0 \xrightarrow{\text{U_ALL}} q_{\text{union}}$
SELECT	$\{q_0, q_{\text{prefix}}, q_{\text{sqli}}\}$	ACEPTA

Resultado: **AMENAZA** — UNION Injection confirmado.

6.1.3. Caso P3 — Consulta legítima (no debe activar alarma)

Consulta SQL:

```
SELECT username, email FROM accounts WHERE user_id = 42
```

Secuencia de tokens:

SELECT, IDENT, IDENT, FROM, IDENT, WHERE, IDENT, EQUALS, NUMBER

El autómata alcanza q_{val} pero no recibe ningún token que provoque transición a q_{sqli} . La cadena termina en $\{q_0, q_{\text{val}}\} - q_{\text{val}} \notin F$.

Resultado: **BENIGNA** — sin amenaza detectada. ✓

6.2. Pruebas sobre Privilege Escalation

Perfil: pgsql-privesc-v1.

6.2.1. Caso P4 — Escalada directa

Consulta: ALTER USER administrador SUPERUSER

Tokens: ALTER, USER, IDENT, SUPERUSER

Token	Estados activos	
ALTER	$\{q_0, q_{\text{alter}}\}$	
USER	$\{q_0, q_{\text{target}}\}$	
IDENT	$\{q_0, q_{\text{name}}\}$	
SUPERUSER	$\{q_0, q_{\text{super}}\}$	ACEPTA

Resultado: **AMENAZA CRÍTICA** — Privilege Escalation.

6.2.2. Caso P5 — Con cláusula WITH

Consulta: ALTER ROLE devuser WITH SUPERUSER

Tokens: ALTER, ROLE, IDENT, WITH, SUPERUSER

Traza: $q_0 \rightarrow q_{\text{alter}} \rightarrow q_{\text{target}} \rightarrow q_{\text{name}} \rightarrow q_{\text{with}} \rightarrow q_{\text{super}}$.

Resultado: **AMENAZA CRÍTICA** detectada también por la ruta alternativa WITH.

6.3. Pruebas sobre Data Exfiltration

Perfil: pgsq1-exfil-v1.

6.3.1. Caso P6 — Volcado masivo

Consulta: SELECT * FROM clientes

Tokens: SELECT, STAR, FROM, IDENT

Ningún token en *absent* = {WHERE, LIMIT} presente. Autómata alcanza q_{ex} .

Resultado: **AMENAZA** — Data Exfiltration.

6.3.2. Caso P7 — SELECT * legítimo con WHERE

Consulta: SELECT * FROM pedidos WHERE estado='activo' LIMIT 100

Tokens incluyen **WHERE** y **LIMIT**. El autómata alcanza q_{ex} pero la restricción negativa invalida la aceptación.

Resultado: **BENIGNA** — restricción negativa activa. ✓

6.4. Resumen de Cadenas de Prueba

Cuadro 6.1: Resumen de resultados de pruebas

ID	Perfil	Patrón	Esperado	Obtenido
P1	SQLI	Tautología OR 1=1	AMENAZA	✓
P2	SQLI	UNION ALL SELECT pg_shadow	AMENAZA	✓
P3	SQLI	Consulta legítima con WHERE	BENIGNA	✓
P4	PRIVESC	ALTER USER SUPERUSER directo	AMENAZA	✓
P5	PRIVESC	ALTER ROLE WITH SUPERUSER	AMENAZA	✓
P6	EXFIL	SELECT * FROM sin WHERE	AMENAZA	✓
P7	EXFIL	SELECT * con WHERE y LIMIT	BENIGNA	✓

Todos los casos producen el resultado esperado, validando la corrección de los autómatas y la eficacia de los mecanismos auxiliares (tautología, restricción negativa, modo substring).

Capítulo 7

Conclusiones

7.1. Síntesis del sistema

LogGuard V2 implementa un sistema de detección de intrusiones en tiempo real para PostgreSQL basado en la teoría formal de autómatas finitos. La arquitectura puede resumirse en tres pilares:

1. **Análisis léxico en dos niveles:** el parseo de logs de PostgreSQL produce entradas estructuradas; la tokenización SQL de cuatro fases transforma sentencias SQL en secuencias sobre un alfabeto controlado de más de 70 tokens.
2. **Detección mediante NFA:** seis perfiles NFA serializados en JSON reconocen subcadenas de tokens que corresponden a patrones de ataque conocidos (SQL Injection, Exfiltración, Escalada de Privilegios, Fuerza Bruta, Descubrimiento y Inyección Temporal).
3. **Arquitectura reactiva:** un pipeline asíncrono basado en `Channel<T>` permite monitorear logs con baja latencia sin bloquear la interfaz gráfica.

7.2. Fortalezas del diseño

- *Extensibilidad:* nuevos vectores de ataque se incorporan añadiendo archivos JSON sin modificar el código del motor.
- *Resistencia a evasión:* la normalización (Fase 1) y la detección semántica de tautologías (Fase 2) contrarrestan técnicas comunes de ofuscación SQL.
- *Rendimiento predecible:* la complejidad lineal $O(|w| \cdot |Q|)$ del simulador de subconjuntos garantiza tiempos de respuesta acotados independientemente del contenido de la consulta.
- *Precisión ajustable:* la restricción negativa reduce falsos positivos sin aumentar la complejidad del autómata.

7.3. Limitaciones y trabajo futuro

- *Cobertura de vocabulario*: el sistema reconoce dialecto PostgreSQL. Extensión a otros dialectos (MySQL, MSSQL, Oracle) requeriría perfiles NFA adicionales y ajustes al tokenizador.
- *Ataques de contexto múltiple*: ataques que distribuyen el payload en múltiples consultas (e.g., inyección por batches) no son detectados por los perfiles actuales, que operan consulta por consulta.
- *Aprendizaje automático*: los perfiles NFA son diseñados manualmente. Un sistema de inferencia automática de autómatas (e.g., L^*) a partir de tráfico etiquetado aumentaría la cobertura de detección.
- *Explotación de SLEEP*: la detección de inyección basada en tiempo mediante el token `SLEEP` no considera el tiempo real de ejecución registrado en las líneas `duration:`, que sería un indicador más preciso.

7.4. Conclusión final

LogGuard V2 demuestra que la teoría formal de autómatas tiene aplicación directa y práctica en sistemas de seguridad modernos. La elección de NFA como motor de detección ofrece un balance óptimo entre expresividad formal, rendimiento computacional y mantenibilidad operacional. El sistema es capaz de detectar en tiempo real las principales clases de ataques SQL sobre PostgreSQL, con tasas de falsos positivos bajas gracias a los mecanismos de restricción negativa y normalización semántica.